

Lecture 7: Quadrotor Control

Lecturer: Luca Carlone

Scribes: Luca Carlone, Markus Ryll

Disclaimer: These notes have not been subjected to the usual scrutiny reserved for formal publications. They may be distributed outside this class only with the permission of the Instructor(s).

This lecture introduces the principles of two standard and well-known quadrotor controllers. First, we discuss an almost globally-stable *geometric controller*, which provides a state-of-the-art approach to perform aggressive maneuvers with quadrotors. The second controller is a lightweight cascaded Proportional Derivative (PD) controller, which works well near hovering. Reading the section about the near-hovering controller is optional and such controller is only reviewed as an example of “old-style” quadrotor control design.

In the following we will refer to the variable names depicted in Fig. 7.1.

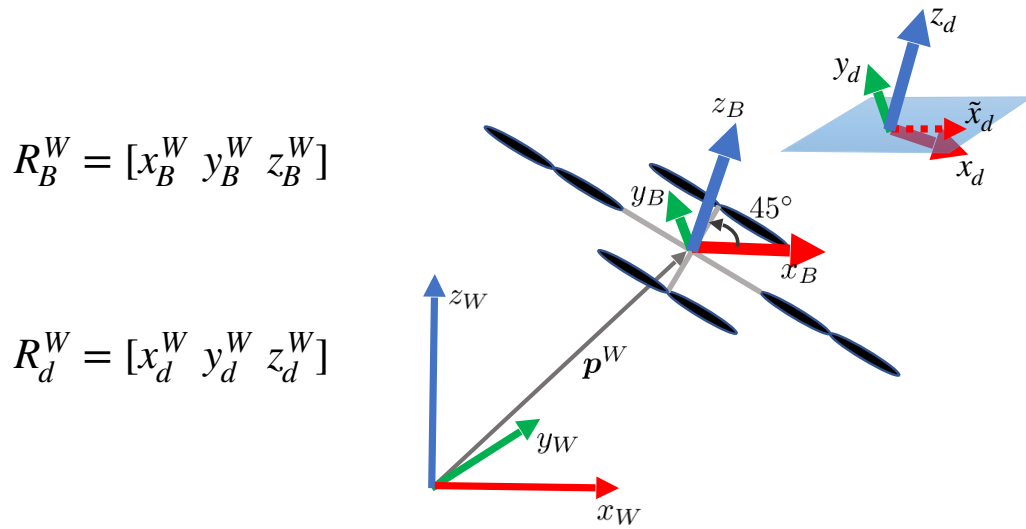


Figure 7.1: Quadrotor - most important variables are labeled. The figure shows the world frame w , the current quadrotor body frame B , and the desired quadrotor frame d (this is where we want our drone to go to). Note that we cannot simultaneously control the position/velocity and roll and pitch, so we only attempt to track a desired yaw direction (indicated by the vector $\tilde{x}_d$), rather than the full attitude R_d^w . The rotations on the left are a reminder that the columns of the rotation R_B^w correspond to the axis of the body frame (e.g., x_B) expressed in the reference frame w (hence x_B^w); the same holds for the rotation R_d^w .

7.1 Notation and definitions

In the rest of these lecture notes, the subscript $[\cdot]_d$ will indicate that $[\cdot]_d$ is the desired value of $[\cdot]$, e.g. p_d^w refers to the desired position while p^w is the current position (in the world frame).

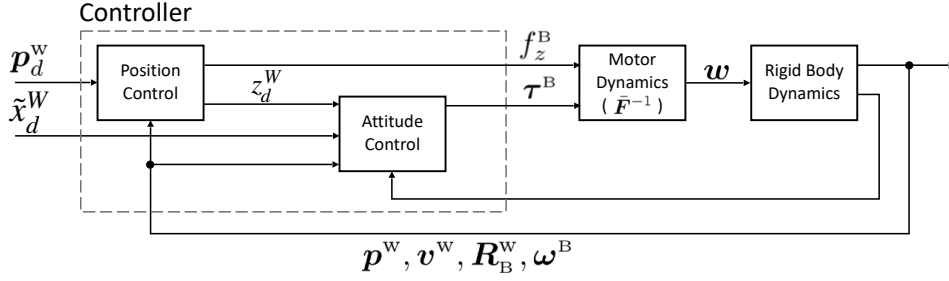


Figure 7.2: The geometric control loops for position and attitude control.

Also in this lecture we will use the common model with the reference frame rotated by 45° with respect to the quadrotor propeller axes (see Fig. 7.1). This definition is more common as we can align the x-axis with a camera frame (with the camera standing between two quadrotor arms), in a way that is not occluded by the quadrotor arms. This is the same model used in the Lab 3 exercises.

7.2 Geometric Controller

This section reviews the geometric controller presented in [3] that we adapt to our reference frame conventions (Fig. 7.1).

A quadrotor has 4 control inputs (the speed of the 4 rotors) but has 6 degrees of freedom (3 for its translation and 3 for its rotation). Since we have less inputs than controls, we will not be able to control independently all the degrees of freedom. Intuitively, if the quadrotor has a pitch or roll angle, its dynamics will force it to move, hence the quadrotor cannot simultaneously keep a desired position and a desired roll and pitch. Therefore, in this lecture we will only discuss how to control 4 degrees of freedom, and in particular the quadrotor position $\mathbf{p}^w$ and the yaw angle ψ . We denote with $\mathbf{p}_d^w$ and ψ_d the desired position and the desired yaw. We equivalently define the yaw angle with a vector in the horizontal plane $\tilde{\mathbf{x}}_d^w = [\cos(\psi) \sin(\psi) 0]^T$.

In the following, we first present the control law and then provide some intuition about its effectiveness.

7.2.1 Tracking Errors and Control Design

In Lecture 6 we have seen that the quadrotor dynamics satisfy:

$$\begin{bmatrix} m\ddot{\mathbf{p}}^w \\ \mathcal{J}\dot{\boldsymbol{\omega}}^B \end{bmatrix} = \begin{bmatrix} -mge_3 \\ -\boldsymbol{\omega}^B \times \mathcal{J}\boldsymbol{\omega}^B \end{bmatrix} + \begin{bmatrix} \mathbf{R}_B^w & \mathbf{0} \\ \mathbf{0} & \mathbf{I}_3 \end{bmatrix} \mathbf{F} \mathbf{w} \quad (7.1)$$

Noting that the first two rows of $\mathbf{F}$ are always zero in our quadrotor model, we rewrite eq. (6.9) in Lecture 6 as follows:

$$\begin{bmatrix} f_z^B \\ \boldsymbol{\tau}^B \end{bmatrix} = \bar{\mathbf{F}} \mathbf{w} \quad (7.2)$$

where $\bar{\mathbf{F}}$ is now a 4×4 invertible matrix. Since $\bar{\mathbf{F}}$ is constant, we can always move back and forth from $\mathbf{w}$ to $[f_z^B; \boldsymbol{\tau}^B]$ and for this reason in the following we will directly think about $[f_z^B; \boldsymbol{\tau}^B]$ as our control inputs and write (7.1) as:

$$\begin{bmatrix} m\ddot{\mathbf{p}}^w \\ \mathcal{J}\dot{\boldsymbol{\omega}}^B \end{bmatrix} = \begin{bmatrix} -mge_3 \\ -\boldsymbol{\omega}^B \times \mathcal{J}\boldsymbol{\omega}^B \end{bmatrix} + \begin{bmatrix} \mathbf{z}_B^w & \mathbf{0} \\ \mathbf{0} & \mathbf{I}_3 \end{bmatrix} \begin{bmatrix} f_z^B \\ \boldsymbol{\tau}^B \end{bmatrix} \quad (7.3)$$

7.2.2 Tracking Errors

One might think about the geometric controller as a PD (Proportional Derivative) controller, but with error metrics that respect the Lie group structure of the rotations. The errors are defined as follows:

$$\mathbf{e}_p = \mathbf{p}^w - \mathbf{p}_d^w \quad (\text{position error}) \quad (7.4)$$

$$\mathbf{e}_v = \mathbf{v}^w - \mathbf{v}_d^w \quad (\text{linear velocity error}) \quad (7.5)$$

$$\mathbf{e}_R = \frac{1}{2} [(\mathbf{R}_d^w)^\top \mathbf{R}_B^w - (\mathbf{R}_B^w)^\top \mathbf{R}_d^w]^\vee \quad (\text{rotation error}) \quad (7.6)$$

$$\mathbf{e}_\omega = \boldsymbol{\omega}^B - (\mathbf{R}_B^w)^\top \mathbf{R}_d^w \boldsymbol{\omega}_d \quad (\text{angular velocity error}) \quad (7.7)$$

The position and (linear) velocity errors are quite straightforward and indeed resemble what one would use in a standard PD controller; again $\mathbf{p}_d^w$ and $\mathbf{v}_d^w$ are the desired position and velocity which is assumed to be given¹. However, the expressions of the rotation and angular velocity errors are more involved. While, we are going to provide some intuitions about these tracking error in the following paragraph, we can think about the rotation and angular velocity errors as carefully-designed vectors where $\mathbf{e}_R$ measures the mismatch between the current quadrotor rotation $\mathbf{R}_B^w$ and the desired rotation $\mathbf{R}_d^w$, and where $\mathbf{e}_\omega$ measures the mismatch between the current body-frame quadrotor angular velocity $\boldsymbol{\omega}^B$ and the desired body-frame angular velocity $\boldsymbol{\omega}_d$.

Intuition for the rotation error: Let us discuss the intuition behind the rotation error in (7.6). In the Lie group lectures, we mentioned that the angular distance between two rotations $\mathbf{R}_B^w$ and $\mathbf{R}_d^w$ can be computed as:

$$\text{dist}_\theta(\mathbf{R}_d^w, \mathbf{R}_B^w) = \|\log((\mathbf{R}_d^w)^\top \mathbf{R}_B^w)^\vee\| \quad (7.8)$$

From Lecture 4, we know the logmap for SO(3) can be simply computed as (see, e.g., https://en.wikipedia.org/wiki/Axis%E2%80%93angle_representation):

$$\log(\mathbf{R}) = \begin{cases} \mathbf{0} & \text{if } \mathbf{R} = \mathbf{I}_3 \\ \frac{\theta}{2 \sin \theta} [\mathbf{R} - \mathbf{R}^\top] & \text{otherwise} \end{cases} \quad (7.9)$$

where θ is the rotation angle.

Therefore, we can develop the log inside the angular distance (7.8) as:

$$\log((\mathbf{R}_d^w)^\top \mathbf{R}_B^w)^\vee = \frac{\theta}{\sin \theta} [(\mathbf{R}_d^w)^\top \mathbf{R}_B^w - (\mathbf{R}_B^w)^\top \mathbf{R}_d^w]^\vee \quad (7.10)$$

from which it should be clear that the rotation error in (7.6) is simply a vector parametrization of the relative error between the desired rotation $\mathbf{R}_d^w$ and the actual rotation $\mathbf{R}_B^w$ (up to scale).

Intuition for the angular velocity error: The intuition behind the angular velocity error in (7.7) is even simpler. The basic observation is that the desired angular velocity $\boldsymbol{\omega}_d$ and the current angular velocity $\boldsymbol{\omega}^B$ live in two different reference frames. The vector $\boldsymbol{\omega}^B$ lives in the body frame of the quadrotor, while $\boldsymbol{\omega}_d$ lives in the desired body frame of the quadrotor (cf. Fig. 7.1). For these two quantities to be comparable, we should express them in the same reference frame. In (7.7), the term $(\mathbf{R}_B^w)^\top \mathbf{R}_d^w \boldsymbol{\omega}_d$ has the effect of first expressing $\boldsymbol{\omega}_d$ in the world frame “w” (via the multiplication by $\mathbf{R}_d^w$) and then transform it to the current body frame “B” (via the multiplication by $(\mathbf{R}_B^w)^\top = \mathbf{R}_B^B$).

¹In Lecture 9-11 we will discuss how to obtain the desired positions and velocities using trajectory optimization

7.2.3 Controller

We can now introduce the geometric controller. For given desired state $\mathbf{p}_d^w, \tilde{\mathbf{x}}_d^w$ (and the corresponding tracking errors $\mathbf{e}_p, \mathbf{e}_v, \mathbf{e}_R, \mathbf{e}_\omega$), and some positive constants k_p, k_v, k_R, k_ω , the control inputs $f_z^B, \boldsymbol{\tau}^B$ are chosen as follows:

$$f_z^B = (-k_p \mathbf{e}_p - k_v \mathbf{e}_v + m \mathbf{g} \mathbf{e}_3 + m \ddot{\mathbf{p}}_d^w) \cdot \mathbf{R}_B^w \mathbf{e}_3, \quad (7.11)$$

$$\boldsymbol{\tau}^B = -k_R \mathbf{e}_R - k_\omega \mathbf{e}_\omega + \boldsymbol{\omega}^B \times \mathcal{J} \boldsymbol{\omega}^B - \mathcal{J}([\boldsymbol{\omega}^B]_\times (\mathbf{R}_B^w)^\top \mathbf{R}_d^w \boldsymbol{\omega}_d - (\mathbf{R}_B^w)^\top \mathbf{R}_d^w \dot{\boldsymbol{\omega}}_d), \quad (7.12)$$

where the desired attitude $\mathbf{R}_d^w$ is computed as $\mathbf{R}_d^w = [\frac{(\mathbf{z}_d^w \times \tilde{\mathbf{x}}_d^w) \times \mathbf{z}_d^w}{\|(\mathbf{z}_d^w \times \tilde{\mathbf{x}}_d^w) \times \mathbf{z}_d^w\|}, \frac{\mathbf{z}_d^w \times \tilde{\mathbf{x}}_d^w}{\|\mathbf{z}_d^w \times \tilde{\mathbf{x}}_d^w\|}, \mathbf{z}_d^w] \in \text{SO}(3)$, and:

$$\mathbf{z}_d^w = \mathbf{R}_d^w \mathbf{e}_3 = \frac{-k_p \mathbf{e}_p - k_v \mathbf{e}_v + m \mathbf{g} \mathbf{e}_3 + m \ddot{\mathbf{p}}_d^w}{\| -k_p \mathbf{e}_p - k_v \mathbf{e}_v + m \mathbf{g} \mathbf{e}_3 + m \ddot{\mathbf{p}}_d^w \|}, \quad (7.13)$$

Here, we assume that the denominator of (7.13) is non-zero,

$$\| -k_p \mathbf{e}_p - k_v \mathbf{e}_v + m \mathbf{g} \mathbf{e}_3 + m \ddot{\mathbf{p}}_d^w \| \neq 0, \quad (7.14)$$

and that the desired trajectory satisfies

$$\| -m \mathbf{g} \mathbf{e}_3 + m \ddot{\mathbf{p}}_d^w \| < B \quad (7.15)$$

for a given positive constant B .

The control moment $\boldsymbol{\tau}^B$ given in (7.12) corresponds to a tracking controller on $\text{SO}(3)$. For the attitude dynamics of a rigid body described by eq. (7.3), (in both the rotational part) this controller exponentially stabilizes the zero equilibrium of the attitude tracking errors. Similarly, the expression in the parentheses in (7.11) corresponds to a tracking controller for the translational dynamics on $\mathbb{R}^3$. The total thrust f_z^B and the desired direction $\mathbf{z}_d^w$ of the third body-fixed axis are chosen so that –if there is no attitude tracking error– the control input term $f_z^B \mathbf{R}_B^w \mathbf{e}_3$ reduces the position and velocity errors. Therefore, the trajectory tracking error will converge to zero provided that the attitude tracking error is identically zero. Certainly, the attitude tracking error may not be zero at any instant. As the attitude tracking error increases, the direction of the control input term $f_z^B \mathbf{R}_B^w \mathbf{e}_3$ of the translational dynamics deviates from the desired direction of $\mathbf{R}_d^w \mathbf{e}_3$. This may cause instability on the complete dynamics. In (7.11), we carefully design the total thrust f_z^B so that its magnitude is reduced when there is a larger attitude tracking error. The expression of f_z^B includes the dot product of the desired third body-fixed axis $\mathbf{z}_d^w = \mathbf{R}_d^w \mathbf{e}_3$ and the current third body-fixed axis $\mathbf{z}_B^w = \mathbf{R}_B^w \mathbf{e}_3$. Therefore, the magnitude of f_z^B is reduced when the angle between those two axes becomes larger. A stability proof can be found in [3].

Intuition about the design of the thrust f_z^B and the desired rotation $\mathbf{R}_d^w$: Let us consider the translational dynamics in eq. (7.3):

$$m \ddot{\mathbf{p}}^w = -m \mathbf{g} \mathbf{e}_3 + \mathbf{R}_B^w \mathbf{f}^B \quad (7.16)$$

Of course, if we compare (7.16) with (7.3) we realize that we can only apply a force $\mathbf{f}^B$ along the local $\mathbf{z}$ of the quadrotor, but let's put that aside for a moment. In the ideal case where we had the freedom to apply forces in any direction, an ideal force to apply would be:

$$\mathbf{f}_{\text{ideal}}^B = \mathbf{R}_B^w (-k_p \mathbf{e}_p - k_v \mathbf{e}_v + m \mathbf{g} \mathbf{e}_3 + m \ddot{\mathbf{p}}_d^w) \quad (7.17)$$

since when we substitute such force into (7.16) we get:

$$m \ddot{\mathbf{p}}^w = -m \mathbf{g} \mathbf{e}_3 - k_p \mathbf{e}_p - k_v \mathbf{e}_v + m \mathbf{g} \mathbf{e}_3 + m \ddot{\mathbf{p}}_d^w \iff \quad (7.18)$$

$$m \ddot{\mathbf{p}}^w = -k_p \mathbf{e}_p - k_v \mathbf{e}_v + m \ddot{\mathbf{p}}_d^w \iff \quad (7.19)$$

$$\text{(using } \mathbf{e}_v = \dot{\mathbf{e}}_p \text{ and } \ddot{\mathbf{p}}^w - \ddot{\mathbf{p}}_d^w = \ddot{\mathbf{e}}_p) \quad (7.20)$$

$$m \ddot{\mathbf{e}}_p = -k_p \mathbf{e}_p - k_v \dot{\mathbf{e}}_p \quad (7.21)$$

which is now a simple linear system describing the dynamics of the position errors. By suitable tuning of the gains k_p and k_v we can ensure that this system converges to zero position (and velocity) error.

Since the drone dynamics restrict the thrust to be along the local body $\mathbf{z}_B^w$ of the quadrotor, the basic idea of the geometric controller is to project the “ideal” thrust force (7.22) along the local body $\mathbf{z}_B^w$ to get:

$$\mathbf{f}_z^B = (-k_p \mathbf{e}_p - k_v \mathbf{e}_v + m \mathbf{g} \mathbf{e}_3 + m \ddot{\mathbf{p}}_d^w) \cdot \mathbf{z}_B^w = (-k_p \mathbf{e}_p - k_v \mathbf{e}_v + m \mathbf{g} \mathbf{e}_3 + m \ddot{\mathbf{p}}_d^w) \cdot \mathbf{R}_B^w \mathbf{e}_3 \quad (7.22)$$

which matches the thrust design in eq. (7.11).

Note that $\mathbf{f}_z^B$ matches the “ideal” force $\mathbf{f}_{\text{ideal}}^B$ when the local body $\mathbf{z}_B^w$ points in the direction of $-k_p \mathbf{e}_p - k_v \mathbf{e}_v + m \mathbf{g} \mathbf{e}_3 + m \ddot{\mathbf{p}}_d^w$. This is indeed the rationale for choosing this vector as the desired local body $\mathbf{z}_d^w$ in (7.13). The desired local body $\mathbf{x}_d^w$ and $\mathbf{y}_d^w$ are built such that the yaw angle matches the desired yaw angle $\hat{\mathbf{x}}_d^w$, see Fig. 7.1, and the axis are orthogonal.

Intuition about the design of the torque $\boldsymbol{\tau}^B$: Similarly to the previous case, we can substitute the torque $\boldsymbol{\tau}^B$ in eq. (7.12) back into the rotational dynamics (7.3):

$$\mathcal{J} \dot{\boldsymbol{\omega}}^B = -\boldsymbol{\omega}^B \times \mathcal{J} \boldsymbol{\omega}^B - k_R \mathbf{e}_R - k_\omega \mathbf{e}_\omega + \boldsymbol{\omega}^B \times \mathcal{J} \boldsymbol{\omega}^B + \mathcal{J}(-[\boldsymbol{\omega}^B]_\times (\mathbf{R}_B^w)^\top \mathbf{R}_d^w \boldsymbol{\omega}_d + (\mathbf{R}_B^w)^\top \mathbf{R}_d^w \dot{\boldsymbol{\omega}}_d) \iff (7.23)$$

$$\mathcal{J} \dot{\boldsymbol{\omega}}^B = -k_R \mathbf{e}_R - k_\omega \mathbf{e}_\omega + \mathcal{J}(-[\boldsymbol{\omega}^B]_\times (\mathbf{R}_B^w)^\top \mathbf{R}_d^w \boldsymbol{\omega}_d + (\mathbf{R}_B^w)^\top \mathbf{R}_d^w \dot{\boldsymbol{\omega}}_d) \quad (7.24)$$

Now we observe that:

$$\dot{\mathbf{e}}_\omega = \frac{d}{dt}(\boldsymbol{\omega}^B - (\mathbf{R}_B^w)^\top \mathbf{R}_d^w \boldsymbol{\omega}_d) = \dot{\boldsymbol{\omega}}^B - (\dot{\mathbf{R}}_B^w)^\top \mathbf{R}_d^w \boldsymbol{\omega}_d - (\mathbf{R}_B^w)^\top \dot{\mathbf{R}}_d^w \boldsymbol{\omega}_d - (\mathbf{R}_B^w)^\top \mathbf{R}_d^w \dot{\boldsymbol{\omega}}_d = \quad (7.25)$$

$$\dot{\boldsymbol{\omega}}^B - (\mathbf{R}_B^w [\boldsymbol{\omega}^B]_\times)^\top \mathbf{R}_d^w \boldsymbol{\omega}_d - (\mathbf{R}_B^w)^\top \mathbf{R}_d^w [\boldsymbol{\omega}_d]_\times \boldsymbol{\omega}_d - (\mathbf{R}_B^w)^\top \mathbf{R}_d^w \dot{\boldsymbol{\omega}}_d = \quad (7.26)$$

$$\text{(recalling that } [\boldsymbol{\omega}^B]^\top = -[\boldsymbol{\omega}^B]_\times \text{ and } [\boldsymbol{\omega}_d]_\times \boldsymbol{\omega}_d = \mathbf{0}) \quad (7.27)$$

$$\dot{\boldsymbol{\omega}}^B + [\boldsymbol{\omega}^B]_\times (\mathbf{R}_B^w)^\top \mathbf{R}_d^w \boldsymbol{\omega}_d - (\mathbf{R}_B^w)^\top \mathbf{R}_d^w \dot{\boldsymbol{\omega}}_d \quad (7.28)$$

Noting that this expression matches terms present in (7.24), eq. (7.24) can be written as:

$$\mathcal{J} \dot{\mathbf{e}}_\omega = -k_R \mathbf{e}_R - k_\omega \mathbf{e}_\omega \quad (7.29)$$

which again is a simple linear system describing the rotation and angular velocity error dynamics. By suitable tuning of the gains k_R and k_ω we can ensure that this system converges to zero rotation and angular velocity error.

7.3 Near-Hovering Controller

This section is optional and is given as an example of how quadrotor controllers were designed before the geometric controller was proposed. The presentation is mostly based on [4].

The aerial robot is controlled by nested feedback loops as shown in Fig. 7.3. The inner attitude control loop usually utilizes an onboard IMU with accelerometers and gyroscopes to control the roll ϕ , pitch θ , and yaw ψ and runs typically at a high frequency (200 Hz – 2 kHz), while the outer position control loop uses estimates of position and velocity of the center of mass to control the trajectory in three dimensions. The controllers are derived by linearizing the equations of motion at an operating point that corresponds to the nominal hover state, $\mathbf{p}^w = \mathbf{p}_0^w$, $\theta = \phi = 0$, $\psi = \psi_0$, $\dot{\mathbf{p}}^w = \mathbf{0}$, and $\dot{\theta} = \dot{\phi} = \dot{\psi} = 0$, where the roll and pitch angles are small ($\cos(\phi) \approx 1$, $\cos(\theta) \approx 1$, $\sin(\phi) \approx \phi$, and $\sin(\theta) \approx \theta$).

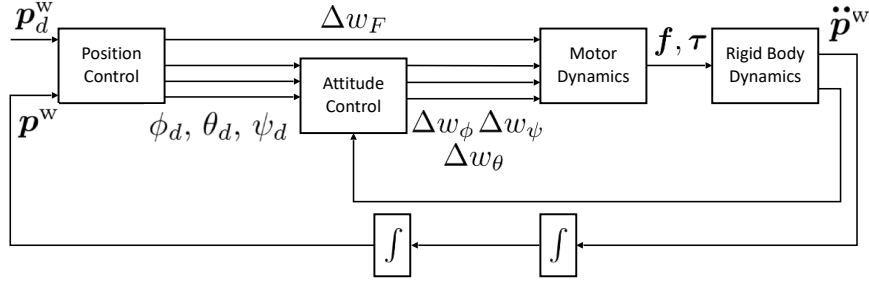


Figure 7.3: The nested control loops for position and attitude control.

At this hover state, the nominal thrusts from the propellers must satisfy

$$\mathbf{T}_{i,0} = -\frac{m\mathbf{g}^w}{4}, \quad (7.30)$$

meaning that the thrust of each propeller is in charge of compensating for 1/4 of the gravity force $m\mathbf{g}^w$.

In order to produce this torque, at hovel, the motor speeds must be set to:

$$\omega_{i,0} = \omega_h = \sqrt{\frac{mg}{4c_f}}, \quad (7.31)$$

where $g = \|\mathbf{g}^w\|$ ($\approx 9.81m/s^2$).

7.3.1 Attitude Control

We will now discuss the attitude controller to track trajectories in $SO(3)$ that are close to the nominal hover state where the roll ϕ and pitch θ angles are small. Using the rotational part of the Newton-Euler equation (see eq. (6.8) in the previous lecture), with angular velocities of the rotors $\mathbf{w} = [w_1|w_1|w_2|w_2|w_3|w_3|w_4|w_4]^T$, yields to

$$\mathcal{J}\dot{\boldsymbol{\omega}}^B = -\boldsymbol{\omega}^B \times \mathcal{J}\boldsymbol{\omega}^B + \boldsymbol{\tau}^B \iff (7.32)$$

$$\mathcal{J}\dot{\boldsymbol{\omega}}^B = -\boldsymbol{\omega}^B \times \mathcal{J}\boldsymbol{\omega}^B + \boldsymbol{\tau}_{\text{drag}}^B + \boldsymbol{\tau}_{\text{thrust}}^B \iff (7.33)$$

$$\mathcal{J}\dot{\boldsymbol{\omega}}^B = -\boldsymbol{\omega}^B \times \mathcal{J}\boldsymbol{\omega}^B + \begin{bmatrix} \sqrt{\frac{1}{2}}c_f\|\boldsymbol{\rho}_1^B\| & \sqrt{\frac{1}{2}}c_f\|\boldsymbol{\rho}_2^B\| & -\sqrt{\frac{1}{2}}c_f\|\boldsymbol{\rho}_3^B\| & -\sqrt{\frac{1}{2}}c_f\|\boldsymbol{\rho}_4^B\| \\ -\sqrt{\frac{1}{2}}c_f\|\boldsymbol{\rho}_1^B\| & \sqrt{\frac{1}{2}}c_f\|\boldsymbol{\rho}_2^B\| & \sqrt{\frac{1}{2}}c_f\|\boldsymbol{\rho}_3^B\| & -\sqrt{\frac{1}{2}}c_f\|\boldsymbol{\rho}_4^B\| \\ -c_d & c_d & -c_d & c_d \end{bmatrix} \begin{bmatrix} w_1|w_1| \\ w_2|w_2| \\ w_3|w_3| \\ w_4|w_4| \end{bmatrix} \quad (7.34)$$

where we assumed that the arms are mounted at 45° as in Fig. 7.1.

Let us call the rotational velocity components $\boldsymbol{\omega}^B = [p, q, r]$. Assuming that $\|\boldsymbol{\rho}_1^B\| = \|\boldsymbol{\rho}_2^B\| = \|\boldsymbol{\rho}_3^B\| = \|\boldsymbol{\rho}_4^B\| = \|\boldsymbol{\rho}^B\|$ and that $\mathcal{J}$ is a diagonal matrix with $\mathcal{J} = \text{diag}[\mathcal{J}_{xx} \mathcal{J}_{yy} \mathcal{J}_{zz}]$, we can reformulate (7.34) as

$$\mathcal{J}_{xx}\dot{p} = \sqrt{\frac{1}{2}}c_f\|\boldsymbol{\rho}^B\|(w_1|w_1| + w_2|w_2| - w_3|w_3| - w_4|w_4|) - qr(\mathcal{J}_{zz} - \mathcal{J}_{yy}) \quad (7.35)$$

$$\mathcal{J}_{yy}\dot{q} = \sqrt{\frac{1}{2}}c_f\|\boldsymbol{\rho}^B\|(-w_1|w_1| + w_2|w_2| + w_3|w_3| - w_4|w_4|) - pr(\mathcal{J}_{xx} - \mathcal{J}_{zz}) \quad (7.36)$$

$$\mathcal{J}_{zz}\dot{r} = c_d(-w_1|w_1| + w_2|w_2| - w_3|w_3| + w_4|w_4|) - pq(\mathcal{J}_{yy} - \mathcal{J}_{xx}). \quad (7.37)$$

The last term in eq. (7.37) is small, since $\mathcal{J}_{xx} \approx \mathcal{J}_{yy}$ because of the symmetry. Additionally, we assume the component of the angular velocity in the $\mathbf{z}_B$ direction, r , is small so the rightmost terms in (7.35) and (7.36) which are products involving r are also small compared to the other terms.

Before using the expressions above, we reparametrize the vector of desired rotor speeds in terms of the nominal motor speed ω_h and the deviations $\Delta\omega_F, \Delta\omega_\phi, \Delta\omega_\theta, \Delta\omega_\psi$:

$$\begin{bmatrix} w_1^d \\ w_2^d \\ w_3^d \\ w_4^d \end{bmatrix} = \begin{bmatrix} 1 & 1 & -1 & -1 \\ 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix} \begin{bmatrix} \omega_h + \Delta\omega_F \\ \Delta\omega_\phi \\ \Delta\omega_\theta \\ \Delta\omega_\psi \end{bmatrix} \iff \begin{bmatrix} \omega_h + \Delta\omega_F \\ \Delta\omega_\phi \\ \Delta\omega_\theta \\ \Delta\omega_\psi \end{bmatrix} = \frac{1}{4} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & -1 & -1 \\ -1 & 1 & 1 & -1 \\ -1 & 1 & -1 & 1 \end{bmatrix} \begin{bmatrix} w_1^d \\ w_2^d \\ w_3^d \\ w_4^d \end{bmatrix} \quad (7.38)$$

where the nominal rotor speed required to hover in steady state is ω_h (see eq. (7.31)), and the deviations from this nominal vector are $\Delta\omega_F, \Delta\omega_\phi, \Delta\omega_\theta$, and $\Delta\omega_\psi$. The term $\Delta\omega_F$ results in a net force along the $\mathbf{z}_B$ axis, while $\Delta\omega_\phi, \Delta\omega_\theta$, and $\Delta\omega_\psi$ produce moments causing roll, pitch, and yaw, respectively.

Now we linearize (7.35), (7.36) and (7.37) about the hovering operating point and write the desired angular accelerations in terms of the new control inputs

$$\dot{p}_d = \frac{4\sqrt{\frac{1}{2}}c_f\|\boldsymbol{\rho}^B\|\omega_h}{\mathcal{J}_{xx}}\Delta\omega_\phi, \quad (7.39)$$

$$\dot{q}_d = \frac{4\sqrt{\frac{1}{2}}\|\boldsymbol{\rho}^B\|\omega_h}{\mathcal{J}_{yy}}\Delta\omega_\theta, \quad (7.40)$$

$$\dot{r}_d = \frac{4c_d\omega_h}{\mathcal{J}_{zz}}\Delta\omega_\psi. \quad (7.41)$$

As near the nominal hover state $\dot{\phi} \approx p, \dot{\theta} \approx q$, and $\dot{\psi} \approx r$, we use proportional derivative control laws that take the form

$$\begin{aligned} \Delta\omega_\phi &= k_{p,\phi}(\phi_d - \phi) + k_{d,\phi}(p_d - p) \\ \Delta\omega_\theta &= k_{p,\theta}(\theta_d - \theta) + k_{d,\theta}(q_d - q) \\ \Delta\omega_\psi &= k_{p,\psi}(\psi_d - \psi) + k_{d,\psi}(r_d - r) . \end{aligned} \quad (7.42)$$

where ϕ_d, θ_d, ψ_d are the desired roll, pitch, and yaw angles. Substituting (7.42) into (7.38) yields the desired rotor speeds.

7.3.2 Position Control

The 3D position controller is used to follow three-dimensional trajectories with modest accelerations so the near-hover assumptions hold. Let's define again, $\mathbf{p}_d^w$ as the desired position and $\mathbf{p}^w$ as the current position (both in the world frame). Let the unit vector tangent to the trajectory at point $\mathbf{p}_d^w$ be $\hat{\mathbf{t}} = \frac{\dot{\mathbf{p}}_d^w}{\|\dot{\mathbf{p}}_d^w\|}$ and the desired velocity vector be $\dot{\mathbf{p}}_d^w$. We define the position and velocity errors as

$$\mathbf{e}_p = ((\mathbf{p}_d^w - \mathbf{p}^w) \cdot \hat{\mathbf{n}})\hat{\mathbf{n}} + ((\mathbf{p}_T^w + \mathbf{p}) \cdot \hat{\mathbf{b}})\hat{\mathbf{b}} \quad (7.43)$$

and

$$\mathbf{e}_v = \dot{\mathbf{p}}_d^w - \dot{\mathbf{p}}^w. \quad (7.44)$$

where $\hat{\mathbf{n}}$ and $\hat{\mathbf{b}}$ are unit vectors orthogonal to $\hat{\mathbf{t}}$. Note that the position error $\mathbf{e}_p$ ignores the error in the tangent direction $\hat{\mathbf{t}}$ and only considers the position error in the normal directions $\hat{\mathbf{n}}$ and $\hat{\mathbf{b}}$.

We calculate the commanded acceleration, $\ddot{\mathbf{p}}$, from PD feedback of the position and velocity errors:

$$\ddot{\mathbf{p}} = k_p \mathbf{e}_p + k_d \mathbf{e}_v + \ddot{\mathbf{p}}_d^w \quad (7.45)$$

This and very similar control approaches have been presented in [1, 2, 4], which also show how to convert this desired acceleration into the desired roll, pitch, and yaw angles ϕ_d, θ_d, ψ_d .

References

- [1] Samir Bouabdallah. Design and control of quadrotors with application to autonomous flying. Technical report, Epfl, 2007.
- [2] Daniel Gurdan, Jan Stumpf, Michael Achtelik, Klaus-Michael Doth, Gerd Hirzinger, and Daniela Rus. Energy-efficient autonomous four-rotor flying robot controlled at 1 khz. In *Robotics and Automation, 2007 IEEE International Conference on*, pages 361–366. IEEE, 2007.
- [3] Taeyoung Lee, Melvin Leoky, and N Harris McClamroch. Geometric tracking control of a quadrotor uav on se (3). In *Decision and Control (CDC), 2010 49th IEEE Conference on*, pages 5420–5425. IEEE, 2010.
- [4] Nathan Michael, Daniel Mellinger, Quentin Lindsey, and Vijay Kumar. The grasp multiple micro-uav testbed. *IEEE Robot. Automat. Mag.*, 17(3):56–65, 2010.